

TechTribers

Java Interview Series · Part 2 · Q75–Q121

Advanced Java Interview Questions

Strings in depth, StringBuilder, regex, exceptions, threads, reflection, java.time and the Java GUI stack, answered the way a senior engineer would answer them.

47	46	22	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `AJ-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**

A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**

What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**

The next question the interviewer is likely to ask, with the answer.

47	7	46	22
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🧵	01	String Fundamentals	Q75–Q80	5
🏗️	02	StringBuilder, StringBuffer & Immutability	Q81–Q86	11
🔎	03	Inspecting & Comparing Strings	Q87–Q97	16
🛠️	04	Transforming & Converting Strings	Q98–Q109	24
🧠	05	Regex, Exceptions, Threads & Reflection	Q110–Q114	33
🌐	06	I/O, Networking & Date-Time	Q115–Q117	39
🖥️	07	Java GUI & Legacy APIs	Q118–Q121	42

© TechTribers. For personal use only. Do not redistribute.

All questions

🧵 String Fundamentals

Q75 What is a String in Java? 5
Q76 What is an immutable String, and why is String immutable? 6
Q77 How do you compare two strings in Java? 7
Q78 How do you concatenate strings in Java? 8
Q79 How do you extract substrings from a string? 9
Q80 What are some common methods of the String class? 9

🏗️ StringBuilder, StringBuffer & Immutability

Q81 What is the StringBuffer class? 11
Q82 What is the StringBuilder class? 12
Q83 What is the difference between String and StringBuffer? 12
Q84 What is the difference between StringBuffer and StringBuilder? 13
Q85 How do you create an immutable class in Java? 14
Q86 What is the StringTokenizer class? 15

🔎 Inspecting & Comparing Strings

Q87 What does the `length()` method return? 16
Q88 What is the purpose of the `charAt()` method? 17
Q89 How do you check if a string is empty? What about `isBlank()`? 17
Q90 Explain the `indexOf()` method with an example. 18
Q91 How does the `lastIndexOf()` method work? 19
Q92 How does the `contains()` method work? 20
Q93 What does the `startsWith()` method do? 20
Q94 What is the purpose of the `endsWith()` method? 21
Q95 How does the `compareTo()` method work? 21
Q96 How is `equalsIgnoreCase()` different from `equals()`? 22
Q97 What is the purpose of the `intern()` method? 23

🛠️ Transforming & Converting Strings

Q98 How do you convert a string to lowercase with `toLowerCase()`? 24
Q99 Explain the `toUpperCase()` method with an example. 25
Q100 How does `trim()` work, and how is it different from `strip()`? 25
Q101 Explain the `replace()` method with an example. 26
Q102 How is `replaceAll()` different from `replace()`? 26
Q103 Explain the `split()` method with an example. 27
Q104 Explain the purpose of `String.join()` with an example. 28
Q105 Explain the `String.format()` method with an example. 29
Q106 What is the purpose of the `toCharArray()` method? 30
Q107 How do you use the `getChars()` method? 30
Q108 What does the `getBytes()` method do? 31
Q109 What is the purpose of the `valueOf()` method? 32

🧠 Regex, Exceptions, Threads & Reflection

Q110 What is a regular expression in Java? Give an example. 33
Q111 What is exception handling in Java? How do you handle exceptions? 34
Q112 What is multithreading? How do you create and start a thread? 35
Q113 What is Java Reflection? How do you inspect classes and invoke methods? 36
Q114 How do you convert one data type to another? Explain casting and parsing. 37

🌐 I/O, Networking & Date-Time

Q115 How do you read user input with the `Scanner` class? 39
Q116 How do you create a simple client-server application with sockets? 40
Q117 How do you work with dates in Java? Why not `Date` and `SimpleDateFormat`? 41

© TechTribers. For personal use only. Do not redistribute.

🖥️ Java GUI & Legacy APIs

Q118	What is AWT? How do you create a simple window with `Frame`?	42
Q119	How do you create a simple Swing application with a button and event handling?	43
Q120	What is JavaFX? How do you create a simple JavaFX app with a button?	44
Q121	What is a Java Applet? Is it still used?	45

© TechTribers. For personal use only. Do not redistribute.

Section 01

String Fundamentals

What a String really is, why it never changes, and the comparison and concatenation rules every interviewer checks.

Questions 75–80 · 6 questions



Q75 AJ-001 • Basic

What is a String in Java?

⚡ Quick answer

`java.lang.String` is an immutable, `final` class representing a sequence of UTF-16 characters. It implements `CharSequence`, `Comparable<String>` and `Serializable`.

Three ways to create one:

- **Literal**: `"Java"` is stored once in the String pool and reused.
- **Constructor**: `new String("Java")` always creates a new heap object.
- **From characters or bytes**: `new String(char[])`, `new String(byte[], charset)`.

Internally, since Java 9 (JEP 254, compact strings) a String is backed by a `byte[]` plus a `coder` flag. Latin-1 text uses 1 byte per character instead of 2, which roughly halves the memory of typical strings.

Java

```
char[] letters = {'t', 'e', 'c', 'h'};
String fromChars = new String(letters);        // "tech"
String literal = "tribers";                    // pooled
String built = fromChars + literal;            // "techtribers"

CharSequence cs = built;                       // String IS-A CharSequence
System.out.println(built + " | " + built.length() + " | " + cs.charAt(0));
```

▶ Output

```
techtribers | 11 | t
```

💡 Pro tip

Strings are the most allocated objects in most Java services. That is why the JVM has compact strings, string deduplication in G1 (`-XX:+UseStringDeduplication`), and a dedicated pool.

© TechTribers. For personal use only. Do not redistribute.

Q76 AJ-002 • Intermediate

What is an immutable String, and why is String immutable?

⚡ Quick answer

Once created, a String's characters can never change. Every "modifying" method returns a new String, and the original stays untouched.

Why the JDK designers made it immutable:

- **Pool safety**: thousands of variables can share one `"OK"` literal only because nobody can change it.
- **Security**: file paths, URLs, class names and DB credentials are passed as Strings. If they could change after validation, checks could be bypassed.
- **Thread safety**: immutable objects can be shared across threads with no locking.
- **Hash caching**: `hashCode()` is computed once and stored, which makes Strings fast `HashMap` keys.



Java

```
String s = "Java";
s.concat(" 21");              // result discarded
System.out.println(s);        // still "Java"

s = s.concat(" 21");          // reassign the reference
System.out.println(s);
```

▶ Output

```
Java
Java 21
```

⚠️ Interview trap

`s.toUpperCase();` on its own line does nothing useful. The return value is the new String; forgetting to assign it is a common bug in screening rounds.

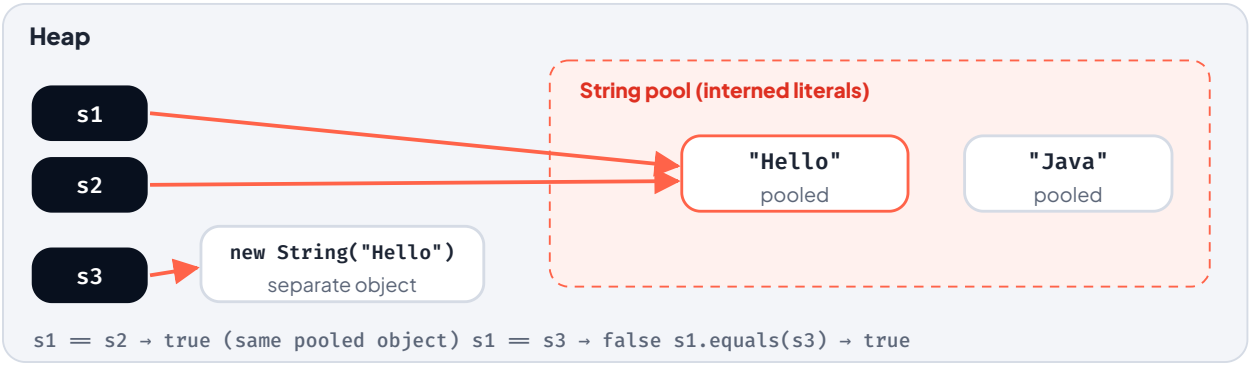
© TechTribers. For personal use only. Do not redistribute.

Q77 AJ-003 • Basic

How do you compare two strings in Java?

⚡ Quick answer

Use `equals()` for content, `equalsIgnoreCase()` for case-insensitive content, `compareTo()` for ordering, and `==` only when you really mean "same object".



Java

```java
String s1 = "Hello";
String s2 = "Hello";
String s3 = new String("Hello");
String input = null;

System.out.println(s1 == s2);                  // true  (same pooled object)
System.out.println(s1 == s3);                  // false (different objects)
System.out.println(s1.equals(s3));             // true  (same content)
System.out.println("Hello".equals(input));     // false, no NPE (literal first)
System.out.println(Objects.equals(input, s1)); // false, null-safe
System.out.println(s1.compareTo("Help"));      // 'l'(108) - 'p'(112) = -4
```

▶ Output

```
true
false
true
false
false
-4
```

⚠️ Interview trap

`s1 == s2` returning `true` for literals tempts people to use `==`. It breaks as soon as one string comes from user input, a DB, `substring()` or `new String()`. Always use `equals()`.

© TechTribers. For personal use only. Do not redistribute.

Q78 AJ-004

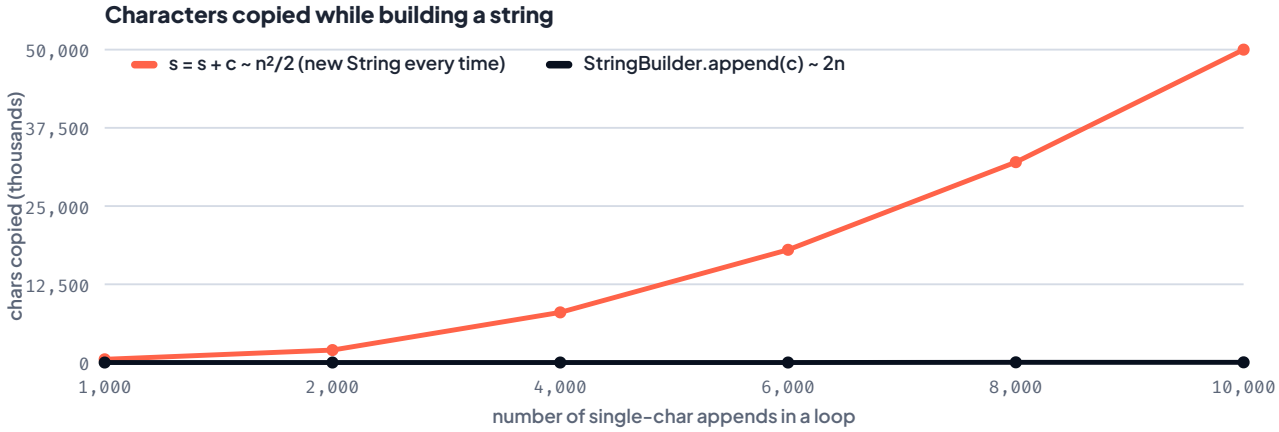
• Basic

How do you concatenate strings in Java?

⚡ **Quick answer**

Use `+` for a few values, `StringBuilder` inside loops, `String.join` or `Collectors.joining` for collections, and `concat()` rarely.

Since Java 9 (JEP 280), `a + b + c` in a single expression compiles to `StringConcatFactory`, which is already optimal. The problem is `+` **inside a loop**: each iteration copies the whole string so far, which is O(n²). The chart shows how fast that grows.



Java

```java
String first = "Asha", last = "Patil";

String a = first + " " + last;                    // compiled to invokedynamic (Java 9+)
String b = first.concat(" ").concat(last);        // NPE if the argument is null
String c = String.join(" ", first, last);
String d = "%s %s".formatted(first, last);        // Java 15+

StringBuilder sb = new StringBuilder();
for (String part : List.of("a", "b", "c")) sb.append(part).append('-');
System.out.println(a + " | " + d + " | " + sb);
```

▶ **Output**

```
Asha Patil | Asha Patil | a-b-c-
```

⚠️ **Interview trap**

`"x" + null` gives `"xnull"`, but `"x".concat(null)` throws `NullPointerException`.

© TechTribers. For personal use only. Do not redistribute.

Q79 AJ-005

• Basic

How do you extract substrings from a string?

⚡ Quick answer

`substring(begin)` returns from `begin` to the end; `substring(begin, end)` returns `begin` inclusive to `end` exclusive. Indexes are zero-based.



Java

```
String msg = "Hello, World!";
System.out.println(msg.substring(0, 5));                    // Hello
System.out.println(msg.substring(7));                       // World!
System.out.println(msg.substring(msg.indexOf(',') + 2));    // World!
System.out.println(msg.substring(5, 5).isEmpty());          // true
// msg.substring(3, 99) -> StringIndexOutOfBoundsException
```

▶ Output

```
Hello
World!
World!
true
```

🎯 Likely follow-up

Did `substring()` ever leak memory? Yes, before Java 7u6 it shared the parent's `char[]`, so a 10-character substring of a 10 MB string kept all 10 MB alive. Since then it copies, so the leak is gone but each call costs O(length).

Q80 AJ-006

• Basic

What are some common methods of the String class?

⚡ Quick answer

Know them by job: inspect, search, compare, transform, split/join, convert. Modern Java added useful ones like `isBlank()`, `strip()`, `repeat()` and `lines()`.

© TechTribers. For personal use only. Do not redistribute.

Job	Methods
Inspect	`length()`, `charAt()`, `isEmpty()`, `isBlank()` (11), `chars()`, `codePoints()`
Search	`indexOf()`, `lastIndexOf()`, `contains()`, `startsWith()`, `endsWith()`, `matches()`
Compare	`equals()`, `equalsIgnoreCase()`, `compareTo()`, `compareToIgnoreCase()`, `contentEquals()`
Transform	`toUpperCase()`, `toLowerCase()`, `trim()`, `strip()` (11), `replace()`, `replaceAll()`, `repeat()` (11), `indent()` (12)
Split / join	`split()`, `String.join()`, `lines()` (11)
Convert	`toCharArray()`, `getBytes()`, `String.valueOf()`, `formatted()` (15), `intern()`

Java

```java
String s = "  Hello, Java  ";
System.out.println(s.strip().toUpperCase());          // HELLO, JAVA
System.out.println(s.isBlank());                      // false
System.out.println("ab".repeat(3));                   // ababab
System.out.println("a\nb\nc".lines().count());        // 3
```

▶ Output

```
HELLO, JAVA
false
ababab
3
```


© TechTribers. For personal use only. Do not redistribute.

Section 02

StringBuilder, StringBuffer & Immutability

Mutable text, thread safety, and how to design your own immutable classes like String.

Questions 81–86 · 6 questions



Q81 AJ-007

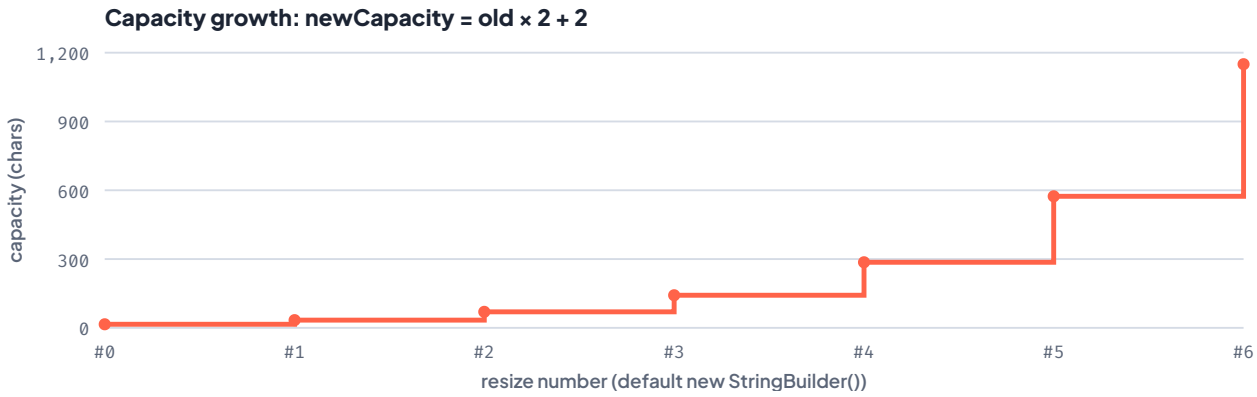
• Basic

What is the StringBuffer class?

⚡ Quick answer

A mutable, **synchronized** sequence of characters (since Java 1.0). Every public method locks the object, so it is thread-safe but slower than `StringBuilder`.

It keeps an internal array with spare **capacity**. When an append doesn't fit, it allocates a bigger array (`old × 2 + 2`) and copies everything across, as the chart shows.



Java

```
StringBuffer buf = new StringBuffer("Hello");   // capacity = 16 + 5 = 21
buf.append(", ").append("Java");
buf.insert(0, ">> ");
buf.replace(3, 8, "Hi");
buf.reverse();
System.out.println(buf + " | capacity=" + buf.capacity());
```

▶ Output

```
avaJ ,iH >> | capacity=21
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> If you know the final size, pass it to the constructor: `new StringBuilder(4096)`. You skip every resize-and-copy step.

⚡ Q82 AJ-008 • Basic

What is the StringBuilder class?

> ⚡ **Quick answer**
> The unsynchronized twin of `StringBuffer`, added in Java 5. Same API, no locking, and the default choice for building strings.

Java

```
static boolean isPalindrome(String s) {
    String clean = s.replaceAll("[^A-Za-z]", "").toLowerCase();
    return new StringBuilder(clean).reverse().toString().equals(clean);
}

StringBuilder csv = new StringBuilder();
for (int i = 1; i <= 3; i++) {
    if (!csv.isEmpty()) csv.append(',');            // isEmpty() on builder: Java 15+
    csv.append("row").append(i);
}
System.out.println(csv + " | " + isPalindrome("Madam, I'm Adam"));
```

▶ **Output**

```
row1,row2,row3 | true
```

> ⚠️ **Interview trap**
> A `StringBuilder` used as a field shared between threads (for example inside a Spring singleton bean) silently corrupts output. Keep builders **method-local**.

VS Q83 AJ-009 • Basic

What is the difference between String and StringBuffer?

> ⚡ **Quick answer**
> `String` is immutable, so every change creates a new object. `StringBuffer` is mutable, so changes happen in place in one object.

String	StringBuffer

© TechTribers. For personal use only. Do not redistribute.

Mutability	Immutable	Mutable
Thread safety	Safe (immutable)	Safe (synchronized)
Pooling	Literals pooled	Never pooled
`equals()`	Compares content	**Not overridden**: compares identity
Use for	Values, keys, constants	Repeated edits shared across threads

Java

```
StringBuffer a = new StringBuffer("tt"), b = new StringBuffer("tt");
System.out.println(a.equals(b));                      // false!
System.out.println(a.toString().equals(b.toString())); // true
System.out.println("tt".contentEquals(a));             // true
```

▶ Output

```
false
true
true
```

⚠️ **Interview trap**

`StringBuffer` and `StringBuilder` do not override `equals()` or `hashCode()`. Never use them as map keys or compare them with `equals()`. Use `contentEquals()` or `compareTo()` (Java 11+).

Q84 AJ-010 • Basic

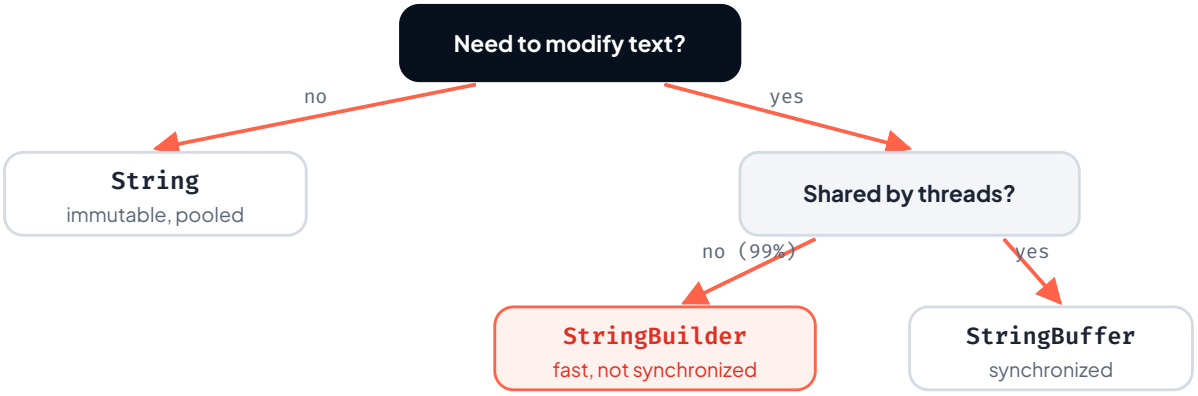
What is the difference between StringBuffer and StringBuilder?

⚡ **Quick answer**

Same API, same internals. `StringBuffer` synchronizes every method; `StringBuilder` doesn't, so it is faster and is the right choice almost always.

	StringBuffer	StringBuilder
Since	Java 1.0	Java 5
Synchronized	Yes, every public method	No
Speed	Lock overhead on every call	Faster
Typical use	Legacy code	Everything else

© TechTribers. For personal use only. Do not redistribute.



💡 **Pro tip**

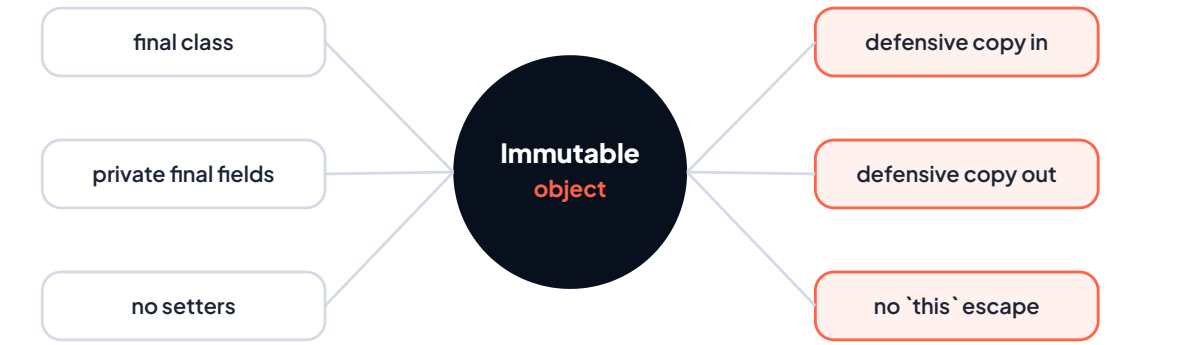
Even when several threads produce text, a shared `StringBuffer` rarely helps: individual `append` calls are atomic, but a sequence of appends from two threads still interleaves. Give each thread its own `StringBuilder` and combine results at the end.

Q85 AJ-011 • Intermediate

How do you create an immutable class in Java?

⚡ **Quick answer**

Make the class `final`, all fields `private final`, provide no setters, and defensively copy any mutable input and output. Since Java 16, records give you most of this for free.




```java
public final class Employee {
    private final String name;
    private final List<String> skills;                // List is mutable!

    public Employee(String name, List<String> skills) {
        this.name = Objects.requireNonNull(name);
```


© TechTribers. For personal use only. Do not redistribute.

```
        this.skills = List.copyOf(skills);            // defensive copy IN
    }
    public String name() { return name; }
    public List<String> skills() { return skills; }    // already unmodifiable: safe OUT

    public Employee withSkill(String s) {              // "wither": returns a new object
        var next = new ArrayList<>(skills); next.add(s);
        return new Employee(name, next);
    }
}

record Point(int x, int y) { }                         // immutable in one line
```

⚠️ **Interview trap**

Records are only **shallowly** immutable. `record Team(List<String> members)` still exposes a mutable list unless you copy it in a compact constructor: `members = List.copyOf(members);`.

🎯 **Likely follow-up**

Why make the class `final`? A subclass could add mutable state or override a getter to return something different, breaking the immutability guarantee callers rely on.

🔪 **Q86** AJ-012 • Basic

What is the StringTokenizer class?

⚡ **Quick answer**

A legacy class (Java 1.0) that breaks a string into tokens using delimiter **characters**. It is kept for compatibility; new code should use `split()`, `Scanner` or regex.

Why it's discouraged:

- Delimiters are individual characters, not a regex or multi-character string.
- It **silently drops empty tokens**, which loses columns when parsing CSV.
- It returns `Object` from the `Enumeration` methods and has no stream support.

Java

```
String line = "Java,,Spring Boot,Kafka";

StringTokenizer st = new StringTokenizer(line, ",");
while (st.hasMoreTokens()) System.out.print("[" + st.nextToken() + "] ");
System.out.println();

System.out.println(Arrays.toString(line.split(",")));   // keeps the empty token
```

▶ **Output**

© TechTribers. For personal use only. Do not redistribute.

```
[Java] [Spring Boot] [Kafka]
[Java, , Spring Boot, Kafka]
```

💡 **Pro tip**

The Javadoc itself calls it a legacy class and recommends `split()` or `java.util.regex`. Saying that in an interview shows you read primary sources.

Section 03

Inspecting & Comparing Strings

Length, characters, searching and comparison, with the edge cases that make each one interview-worthy.

Questions 87–97 · 11 questions



Q87 AJ-013 • Basic

What does the `length()` method return?

⚡ **Quick answer**

The number of UTF-16 code units in the string, which equals the number of characters for most text but not for emoji and other supplementary characters.

`s.length()`	**`arr.length`**	`list.size()`
String: method	array: final field, no ()	Collection: method

Java

```java
System.out.println("Hello, World!".length());    // 13
System.out.println("".length());                 // 0
System.out.println("👋".length());               // 4  (two surrogate pairs)
System.out.println("👋".codePointCount(0, 4));    // 2
```

▶ **Output**

```
13
0
4
2
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
Classic trick question: `length()` for String, `length` (no parentheses) for arrays, `size()` for collections. Mixing them up is a compile error.

Q88 AJ-014 • Basic

What is the purpose of the `charAt()` method?

⚡ **Quick answer**
`charAt(i)` returns the `char` at a zero-based index. An index outside `0..length()-1` throws `StringIndexOutOfBoundsException`.

Java

```java
String word = "Interview";
int vowels = 0;
for (int i = 0; i < word.length(); i++) {
    if ("aeiouAEIOU".indexOf(word.charAt(i)) >= 0) vowels++;
}
long viaStream = word.chars().filter(c -> "aeiouAEIOU".indexOf(c) >= 0).count();
System.out.println(word.charAt(0) + " | " + vowels + " | " + viaStream);
```

▶ **Output**

```
I | 4 | 4
```

⚠️ **Interview trap**
`charAt()` returns a `char`, so `word.charAt(0) + word.charAt(1)` adds two numbers (73 + 110 = 183) instead of joining letters. Start with `"" +` or use `String.valueOf()`.

Q89 AJ-015 • Basic

How do you check if a string is empty? What about `isBlank()`?

⚡ **Quick answer**
`isEmpty()` is true only when `length() == 0`. `isBlank()` (Java 11) is also true for whitespace-only strings. Neither handles `null`.

Input	isEmpty()	isBlank()
`""`	true	true
`" "`	false	true

© TechTribers. For personal use only. Do not redistribute.

`"\t\n"`	false	true
`"a"`	false	false
`null`	NPE	NPE

Java

```
static boolean hasText(String s) {           // null-safe helper
    return s != null && !s.isBlank();
}
System.out.println(hasText(null) + " " + hasText("   ") + " " + hasText("TT"));
```

▶ Output

```
false false true
```

💡 **Pro tip**

In Spring projects use `StringUtils.hasText(s)`; in Apache Commons, `StringUtils.isNotBlank(s)`. Both are null-safe.

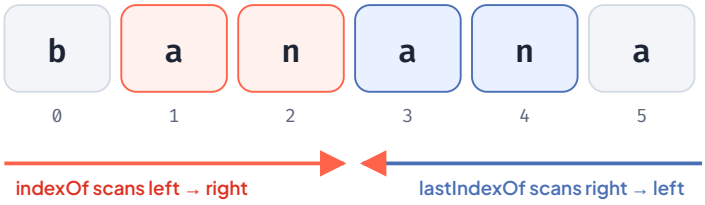
🎯 Q90 AJ-016 • Basic

Explain the `indexOf()` method with an example.

⚡ **Quick answer**

`indexOf()` returns the index of the first occurrence of a char or substring, or `-1` if absent. An overload takes a `fromIndex` to continue searching.

```
indexOf("an") → 1 lastIndexOf("an") → 3 indexOf("an", 2) → 3 indexOf("x") → -1
```



Java

```
String s = "banana";
System.out.println(s.indexOf('a'));        // 1
System.out.println(s.indexOf("an"));       // 1
System.out.println(s.indexOf("an", 2));    // 3
System.out.println(s.indexOf("x"));        // -1

// find every occurrence
```


© TechTribers. For personal use only. Do not redistribute.

```
List<Integer> hits = new ArrayList<>();
for (int i = s.indexOf("a"); i >= 0; i = s.indexOf("a", i + 1)) hits.add(i);
System.out.println(hits);
```

▶ Output

```
1
1
3
-1
[1, 3, 5]
```

💡 **Pro tip**

Complexity is O(n·m) in the worst case. For heavy search over large text, use a `Pattern` or a proper algorithm like KMP or Aho-Corasick.

🔙 **Q91** AJ-017 • Basic

How does the `lastIndexOf()` method work?

⚡ **Quick answer**

`lastIndexOf()` returns the index of the last occurrence, scanning from the end. With `fromIndex` it searches backwards starting at that index.

Java

```
String file = "report.final.v2.pdf";
int dot = file.lastIndexOf('.');
String ext  = dot >= 0 ? file.substring(dot + 1) : "";
String base = dot >= 0 ? file.substring(0, dot) : file;

String path = "/home/tt/docs/notes.md";
String name = path.substring(path.lastIndexOf('/') + 1);

System.out.println(ext + " | " + base + " | " + name);
```

▶ Output

```
pdf | report.final.v2 | notes.md
```

⚠️ **Interview trap**

Using `indexOf('.')` for file extensions breaks on names like `report.final.pdf`. Extensions and file names always need `lastIndexOf`.

© TechTribers. For personal use only. Do not redistribute.

Q92 AJ-018 • Basic

How does the `contains()` method work?

⚡ **Quick answer**

`contains(CharSequence s)` returns `true` if the sequence occurs anywhere in the string. It is case-sensitive and is implemented as `indexOf(s) >= 0`.

Java

```
String log = "ERROR 2026-10-14 Payment timeout for order 42";
System.out.println(log.contains("timeout"));                        // true
System.out.println(log.contains("TIMEOUT"));                        // false
System.out.println(log.toLowerCase(Locale.ROOT).contains("timeout")); // case-insensitive
System.out.println(log.contains(new StringBuilder("order")));       // any CharSequence
```

▶ **Output**

```
true
false
true
true
```

⚠️ **Interview trap**

`contains(null)` throws `NullPointerException`, and `contains("")` is always `true`.

Q93 AJ-019 • Basic

What does the `startsWith()` method do?

⚡ **Quick answer**

`startsWith(prefix)` checks whether the string begins with the prefix. `startsWith(prefix, offset)` checks starting at a given index.

Java

```
String url = "https://techtriber.com/java";
System.out.println(url.startsWith("https://"));          // true
System.out.println(url.startsWith("techtriber", 8));     // true: checks from index 8
System.out.println(url.startsWith(""));                  // true, always

List<String> topics = List.of("java-core", "java-streams", "kafka-basics");
System.out.println(topics.stream().filter(t -> t.startsWith("java")).toList());
```

▶ **Output**

```
true
true
```


© TechTribers. For personal use only. Do not redistribute.

```
true
[java-core, java-streams]
```

Q94 AJ-020 • Basic

What is the purpose of the `endsWith()` method?

⚡ **Quick answer**

`endsWith(suffix)` checks whether the string ends with the suffix. Typical uses: file extensions, domain checks, and validation.

Java

```java
String file = "Invoice_Q3.PDF";
System.out.println(file.endsWith(".pdf"));                          // false: case!
System.out.println(file.toLowerCase(Locale.ROOT).endsWith(".pdf")); // true

String email = "asha@techtriber.com";
System.out.println(email.endsWith("@techtriber.com"));              // true
```

▶ **Output**

```
false
true
true
```

⚠️ **Interview trap**

Domain checks with `endsWith("techtriber.com")` also accept `evil-techtriber.com`. Always include the separator: `"@techtriber.com"` or `".techtriber.com"`.

Q95 AJ-021 • Intermediate

How does the `compareTo()` method work?

⚡ **Quick answer**

It compares strings lexicographically by UTF-16 values. It returns 0 if equal, a negative number if this string sorts first, and a positive number if it sorts after.

The exact value: the difference between the **first differing characters**, or, if one string is a prefix of the other, the **difference in lengths**. Only the sign is guaranteed by the contract, so never rely on the exact number in your logic.

© TechTribers. For personal use only. Do not redistribute.

`"apple".compareTo("banana")`	`'a'(97) - 'b'(98)`	`-1`	first differing char
`"app".compareTo("apple")`	`3 - 5`	`-2`	prefix: length difference
`"Apple".compareTo("apple")`	`'A'(65) - 'a'(97)`	`-32`	uppercase sorts first
`"java".compareTo("java")`	–	`0`	equal

Java

```
List<String> names = new ArrayList<>(List.of("ravi", "Asha", "om", "Zara"));
Collections.sort(names);                                    // natural order
System.out.println(names);
names.sort(String.CASE_INSENSITIVE_ORDER);                  // ignore case
System.out.println(names);
names.sort(Comparator.comparing(String::length).thenComparing(Comparator.naturalOrder()));
System.out.println(names);
```

▶ Output

```
[Asha, Zara, om, ravi]
[Asha, om, ravi, Zara]
[om, Asha, Zara, ravi]
```

⚠️ **Interview trap**

Natural order puts all uppercase letters before lowercase (`Z` < `a`). For human-facing sorting use `String.CASE_INSENSITIVE_ORDER` , or a `java.text.Collator` for locale-aware ordering.

Q96 AJ-022 • Basic

How is `equalsIgnoreCase()` different from `equals()` ?

⚡ **Quick answer**

`equals()` matches exact characters. `equalsIgnoreCase()` treats upper and lower case of each character as equal, without creating new strings.

Java

```
String a = "HeLLo", b = "hello";
System.out.println(a.equals(b));                // false
System.out.println(a.equalsIgnoreCase(b));      // true
System.out.println(a.equalsIgnoreCase(null));   // false, no NPE

// Avoid: allocates two new strings and is locale-sensitive
System.out.println(a.toLowerCase().equals(b.toLowerCase()));
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
false
true
false
true
```

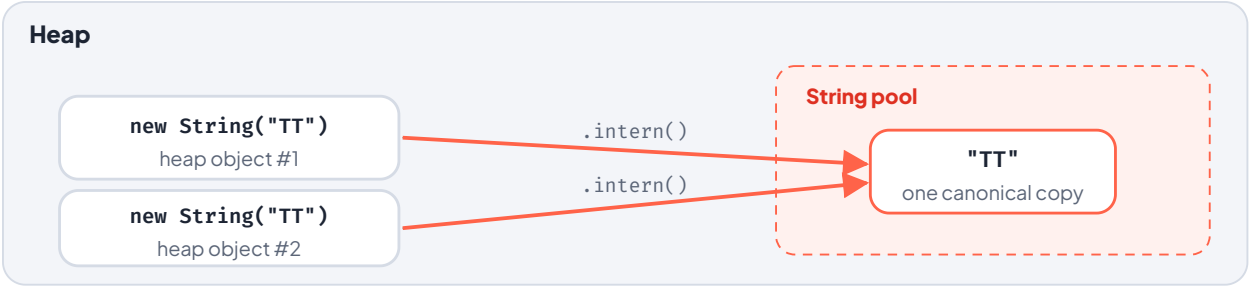
> 💡 **Pro tip**
> For case-insensitive `Set` or `Map` keys, use `new TreeMap<>(String.CASE_INSENSITIVE_ORDER)` instead of lower-casing every key.

Q97 AJ-023 • Intermediate

What is the purpose of the `intern()` method?

> ⚡ **Quick answer**
> `intern()` returns the canonical copy of the string from the String pool, adding it to the pool if it isn't there. Equal interned strings are then the same object, so `==` works.

Since Java 7 the pool lives on the regular heap (not PermGen), so interned strings are garbage-collected like anything else. Size the pool table with `-XX:StringTableSize` if you intern millions of values.



Java

```
String a = new String("TT");
String b = new String("TT");
System.out.println(a == b);                    // false
System.out.println(a.intern() == b.intern());  // true
System.out.println(a.intern() == "TT");        // true: literals are pre-interned
```

▶ Output

```
false
true
true
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Interning to "save memory" usually backfires: the lookup costs CPU on every call, and G1's string deduplication does the job automatically. Use `intern()` only for a small, highly repetitive set of values.

Section 04

Transforming & Converting Strings



Case, whitespace, replace, split, join, format and encoding: the methods with the most production bugs hiding in them.

Questions 98–109 · 12 questions

Q98 AJ-024

• Intermediate

How do you convert a string to lowercase with `toLowerCase()`?

⚡ **Quick answer**

`toLowerCase()` returns a new lower-case string using the **default locale**. For identifiers, keys and protocol values always pass `Locale.ROOT`.

Java

```
String title = "TITLE";
System.out.println(title.toLowerCase(Locale.ROOT));              // title
System.out.println(title.toLowerCase(Locale.forLanguageTag("tr"))); // tıtle (dotless ı)

// The production bug: default locale is Turkish on the server
Locale.setDefault(Locale.forLanguageTag("tr"));
System.out.println("ID".toLowerCase().equals("id"));             // false!
```

▶ **Output**

```
title
tıtle
false
```

⚠️ **Interview trap**

This "Turkish I" bug has broken real systems: config keys, HTTP header names and enum lookups fail only on servers with a Turkish locale. `toLowerCase(Locale.ROOT)` makes it deterministic.

© TechTribers. For personal use only. Do not redistribute.

🔼 Q99 AJ-025

• Intermediate

Explain the `toUpperCase()` method with an example.

⚡ Quick answer

`toUpperCase()` returns a new upper-case string, locale-sensitive like `toLowerCase()`. The result can even be **longer** than the input.

Java

```
System.out.println("hello, world!".toUpperCase(Locale.ROOT));   // HELLO, WORLD!
String german = "straße";
String upper = german.toUpperCase(Locale.ROOT);
System.out.println(upper + " " + german.length() + "->" + upper.length()); // STRASSE 6->7
```

▶ Output

```
HELLO, WORLD!
STRASSE 6->7
```

⚠️ Interview trap

Code that upper-cases a string and then uses indexes from the original string can go out of bounds, because "β" becomes "SS".

🧹 Q100 AJ-026

• Intermediate

How does `trim()` work, and how is it different from `strip()`?

⚡ Quick answer

`trim()` removes leading and trailing characters with code ≤ U+0020 (ASCII space and control chars). `strip()` (Java 11) removes all Unicode whitespace, including non-breaking and em spaces.

Java

```
String ascii   = "   Hello, Java   ";
String unicode = "\u2003Hello\u2003";           // em spaces (common in copy-paste)

System.out.println("[" + ascii.trim() + "]");
System.out.println(unicode.trim().length());      // 7: trim() missed them
System.out.println(unicode.strip().length());     // 5
System.out.println("[" + "  hi  ".stripLeading() + "]" + "[" + "  hi  ".stripTrailing() + "]");
```

▶ Output

```
[Hello, Java]
7
5
[hi  ][  hi]
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> For user input coming from web forms or copy-paste from documents, prefer `strip()`. Hidden Unicode spaces are a frequent cause of "login fails but the password looks right" tickets.

🔁 Q101 AJ-027

• Basic

Explain the `replace()` method with an example.

> ⚡ **Quick answer**
> `replace()` replaces **every** occurrence of a char or a literal `CharSequence` with another. It does not use regex, and it returns a new string.

Java

```
String s = "Hello, World!";
System.out.println(s.replace('o', '0'));          // Hell0, W0rld!
System.out.println(s.replace("World", "Java"));   // Hello, Java!
System.out.println("a.b.c".replace(".", "/"));    // a/b/c  (literal dot)
System.out.println("aaa".replace("aa", "b"));     // ba     (left to right, no overlap)
```

▶ **Output**

```
Hell0, W0rld!
Hello, Java!
a/b/c
ba
```

> ⚠️ **Interview trap**
> Many candidates believe `replace()` only replaces the first match because of the name `replaceAll()`. It replaces **all** occurrences; the difference is regex, not count.

🧪 Q102 AJ-028

• Intermediate

How is `replaceAll()` different from `replace()`?

> ⚡ **Quick answer**
> `replaceAll()` and `replaceFirst()` treat the first argument as a **regex** and the replacement as a template where `$1` means group 1. `replace()` is literal on both sides.

Method	Pattern	Occurrences
`replace(a, b)`	Literal	All
`replaceAll(regex, b)`	Regex	All

© TechTribers. For personal use only. Do not redistribute.

`replaceFirst(regex, b)`	Regex	First only

Java

```java
System.out.println("a.b.c".replaceAll(".", "-"));                // ----- (dot = any char)
System.out.println("a.b.c".replaceAll("\\.", "-"));              // a-b-c
System.out.println("2026-10-14".replaceAll("(\\d+)-(\\d+)-(\\d+)", "$3/$2/$1")); // 14/10/2026
System.out.println("  too   many  ".replaceAll("\\s+", " ").strip());  // too many
String price = "$5";
System.out.println("Cost: X".replace("X", price));               // literal: fine
// "Cost: X".replaceAll("X", price) -> IndexOutOfBoundsException: No group 5
System.out.println("Cost: X".replaceAll("X", Matcher.quoteReplacement(price)));
```

▶ Output

```
-----
a-b-c
14/10/2026
too many
Cost: $5
Cost: $5
```

💡 **Pro tip**

In a loop, `replaceAll` recompiles the regex on every call. Compile once: `private static final Pattern WS = Pattern.compile("\\s+");` then `WS.matcher(s).replaceAll(" ")`.

🪓 Q103 AJ-029 • Intermediate

Explain the `split()` method with an example.

⚡ **Quick answer**

`split(regex)` breaks a string around regex matches and returns an array. Trailing empty strings are removed unless you pass a negative limit.

© TechTribers. For personal use only. Do not redistribute.

```java
System.out.println(Arrays.toString("apple, banana, orange".split(",\\s*")));
System.out.println("a,b,,c,,".split(",").length);          // 4: trailing empties dropped
System.out.println("a,b,,c,,".split(",", -1).length);      // 6: keep everything
System.out.println(Arrays.toString("k=v=x".split("=", 2)));  // limit 2
System.out.println(Arrays.toString("a|b".split("|")));     // regex trap!
System.out.println(Arrays.toString("a|b".split("\\|")));
```

▶ Output

```
[apple, banana, orange]
4
6
[k, v=x]
[a, |, b]
[a, b]
```

⚠️ Interview trap

Characters like `|`, `.`, `*`, `+`, `$` and `?` are regex metacharacters. `split(".")` returns an **empty array** for most input. Escape them (`"\\."`) or use `Pattern.quote(".")`.

🔗 Q104 AJ-030 • Basic

Explain the purpose of `String.join()` with an example.

⚡ Quick answer

`String.join(delimiter, elements...)` (Java 8) joins strings or any `Iterable<CharSequence>` with a delimiter. For prefixes and suffixes use `Collectors.joining` or `StringJoiner`.

```java
List<String> stack = List.of("Java", "Spring Boot", "Kafka");

System.out.println(String.join(", ", stack));
System.out.println(stack.stream().collect(Collectors.joining(" | ", "[", "]")));

StringJoiner sql = new StringJoiner(", ", "IN (", ")");
List.of(101, 102, 103).forEach(id -> sql.add(String.valueOf(id)));
System.out.println(sql);
```

▶ Output

```
Java, Spring Boot, Kafka
[Java | Spring Boot | Kafka]
IN (101, 102, 103)
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Building SQL by joining user values is a SQL-injection hole. The `IN (...)` example is only safe because the IDs are integers you control; for user input use `PreparedStatement` parameters.

📝 Q105 AJ-031 • Basic

Explain the `String.format()` method with an example.

⚡ **Quick answer**

`String.format(pattern, args...)` builds a string from format specifiers like `%s`, `%d`, `%.2f`. Java 15 added the instance form `"...".formatted(args)`.

Specifier	Meaning	Example → output
`%s`	Any object via `toString()`	`"Asha"` → Asha
`%d`	Integer	`42` → 42
`%,d`	Grouped integer	`1234567` → 1,234,567
`%.2f`	Float, 2 decimals	`3.14159` → 3.14
`%05d`	Zero-padded width 5	`42` → 00042
`%-8s|`	Left-aligned width 8	`"Java"` → Java|
`%n`	Platform line separator	new line

Java

```
String row = String.format(Locale.US, "%-8s|%6.2f|%,d", "Kafka", 99.5, 1_234_567);
System.out.println(row);
System.out.println("Order %05d for %s".formatted(42, "Asha"));
```

▶ **Output**

```
Kafka   | 99.50|1,234,567
Order 00042 for Asha
```

💡 **Pro tip**

Pass a `Locale` when output must be stable: with the default German locale, `%.2f` prints `99,50`. For logs, use the logger's `{}` placeholders instead of formatting strings yourself.

© TechTribers. For personal use only. Do not redistribute.

Q106 AJ-032

• Basic

What is the purpose of the `toCharArray()` method?

⚡ **Quick answer**

`toCharArray()` returns a **new** `char[]` copy of the string's characters, which you can sort or modify without affecting the original string.

Java

```
static boolean isAnagram(String a, String b) {
    char[] x = a.replace(" ", "").toLowerCase(Locale.ROOT).toCharArray();
    char[] y = b.replace(" ", "").toLowerCase(Locale.ROOT).toCharArray();
    Arrays.sort(x);
    Arrays.sort(y);
    return Arrays.equals(x, y);
}
System.out.println(isAnagram("Listen", "Silent"));        // true
System.out.println(isAnagram("Kafka", "Flink"));          // false
```

▶ **Output**

```
true
false
```

🎯 **Likely follow-up**

Why do security APIs like `JPasswordField.getPassword()` return `char[]` instead of `String`? A `char[]` can be wiped with `Arrays.fill(pw, '\0')` after use. An immutable String stays in memory until GC, and could end up in heap dumps.

Q107 AJ-033

• Intermediate

How do you use the `getChars()` method?

⚡ **Quick answer**

`getChars(srcBegin, srcEnd, dst, dstBegin)` copies a range of characters into an **existing** `char[]` at a given position, without allocating a new array.

It is the low-level, allocation-free cousin of `toCharArray()`, used in parsers, buffers and the JDK's own `StringBuilder`. Index errors throw `IndexOutOfBoundsException`.

© TechTribers. For personal use only. Do not redistribute.

```java
String s = "Hello, Java";
char[] buf = new char[10];
Arrays.fill(buf, '.');
s.getChars(7, 11, buf, 3);                // copy "Java" into buf starting at index 3
System.out.println(new String(buf));
```

▶ Output

```
...Java...
```

Q108 AJ-034 • Intermediate

What does the `getBytes()` method do?

⚡ Quick answer

`getBytes()` encodes the string into a `byte[]` using a charset. Always pass one explicitly, such as `StandardCharsets.UTF_8`; the no-arg version uses the platform default.

A	UTF-8	41				1 byte
₹	UTF-8	E2	82	B9		3 bytes
₹	UTF-16BE	20	B9			2 bytes
😀	UTF-8	F0	9F	98	80	4 bytes

```java
String s = "Java ₹";
System.out.println(s.getBytes(StandardCharsets.UTF_8).length);     // 8
System.out.println(s.getBytes(StandardCharsets.UTF_16BE).length);  // 12
System.out.println(s.getBytes(StandardCharsets.ISO_8859_1).length); // 6: ₹ becomes '?'

byte[] wire = s.getBytes(StandardCharsets.UTF_8);
System.out.println(new String(wire, StandardCharsets.UTF_8));      // round-trips
```

▶ Output

```
8
12
6
Java ₹
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Before Java 18 the default charset depended on the OS (Windows often used Cp1252), so `getBytes()` gave different bytes on the developer laptop and on the Linux server. Java 18 (JEP 400) made UTF-8 the default, but explicit charsets remain best practice.

🔄 **Q109** `AJ-035` • Basic

What is the purpose of the `valueOf()` method?

⚡ **Quick answer**

`String.valueOf(x)` converts primitives, char arrays and objects to a String. For objects it is null-safe: it returns `"null"` instead of throwing.

Java

```
System.out.println(String.valueOf(42) + 1);          // "421"
System.out.println(String.valueOf(3.5f));            // 3.5
System.out.println(String.valueOf(new char[]{'t','t'})); // tt

Object nothing = null;
System.out.println(String.valueOf(nothing));         // "null"
// nothing.toString()                                -> NullPointerException
// String.valueOf((char[]) null)                     -> NullPointerException!
```

▶ **Output**

```
421
3.5
tt
null
```

⚠️ **Interview trap**

`String.valueOf(null)` with a literal `null` picks the `char[]` overload (the most specific type) and throws `NullPointerException`. With a typed `Object` variable it safely returns `"null"`.

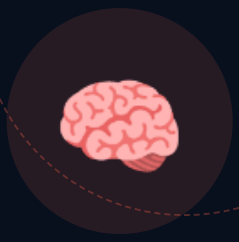
© TechTribers. For personal use only. Do not redistribute.

Section 05

Regex, Exceptions, Threads & Reflection

The core platform APIs behind every framework you use: pattern matching, error handling, concurrency and runtime introspection.

Questions 110–114 · 5 questions



Q110 AJ-036

• Intermediate

What is a regular expression in Java? Give an example.

⚡ Quick answer

A regex is a pattern describing a set of strings. Java's `java.util.regex` has `Pattern` (the compiled, thread-safe pattern) and `Matcher` (a stateful, single-thread search over one input).

`matches()` requires the **whole** input to match; `find()` looks for the next match anywhere; `lookingAt()` matches from the start only.

Pattern: ^[A-Z]{3}-(\d{4})-(\d{3})$ matches "ORD-2026-042"

^	[A-Z]{3}	-	(\d{4})	-	(\d{3})	$
start	3 capitals	literal -	group 1: year	literal -	group 2: seq	end

Java

```
private static final Pattern ORDER_ID =
        Pattern.compile("^[A-Z]{3}-(?<year>\\d{4})-(?<seq>\\d{3})$");

Matcher m = ORDER_ID.matcher("ORD-2026-042");
if (m.matches()) {
    System.out.println(m.group("year") + " / " + m.group("seq"));
}

Matcher f = Pattern.compile("\\d+").matcher("Q3 revenue 450 cr, growth 18%");
List<String> nums = new ArrayList<>();
while (f.find()) nums.add(f.group());
System.out.println(nums);
```

▶ Output

```
2026 / 042
[3, 450, 18]
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Nested quantifiers like `(a+)+` on attacker-controlled input can take exponential time (catastrophic backtracking, ReDoS) and freeze a request thread. Keep patterns simple, limit input length, and test with long non-matching strings.

🚨 **Q111** AJ-037 • Intermediate

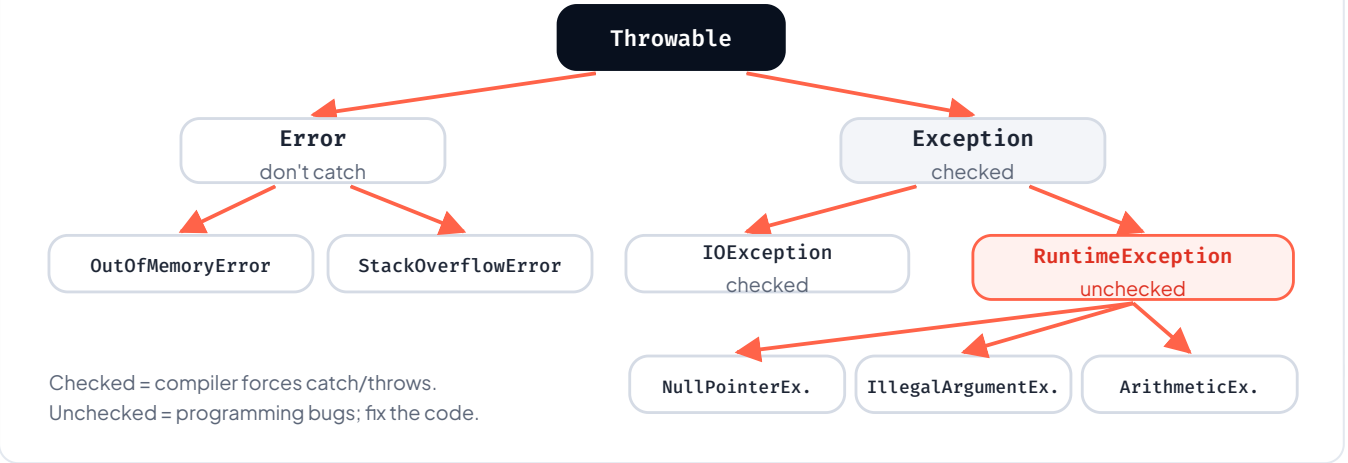
What is exception handling in Java? How do you handle exceptions?

⚡ **Quick answer**

Exception handling separates error handling from normal flow using `try`, `catch`, `finally`, `throw` and `throws`. Checked exceptions must be caught or declared; unchecked ones (`RuntimeException`) need not be.

Rules that come up in interviews:

- `finally` runs whether or not an exception is thrown, except after `System.exit()` or a JVM crash.
- **try-with-resources** (Java 7) closes any `AutoCloseable` automatically, in reverse order, and keeps close failures as *suppressed* exceptions.
- `throw` raises one exception; `throws` declares what a method may raise.



Java

```
class InsufficientFundsException extends Exception {          // custom checked
    InsufficientFundsException(String msg) { super(msg); }
}

static void withdraw(BigDecimal balance, BigDecimal amount) throws InsufficientFundsException {
    if (amount.compareTo(balance) > 0)
        throw new InsufficientFundsException("Need " + amount + ", have " + balance);
}

public static void main(String[] args) {
    try (var in = new BufferedReader(new StringReader("500"))) {    // try-with-resources
```


© TechTribers. For personal use only. Do not redistribute.

```
        withdraw(new BigDecimal(in.readLine()), new BigDecimal("800"));
    } catch (InsufficientFundsException | IOException e) {         // multi-catch
        System.out.println("Failed: " + e.getMessage());
    } finally {
        System.out.println("Audit logged");
    }
}
```

▶ Output

```
Failed: Need 800, have 500
Audit logged
```

⚠️ **Interview trap**

Never `return` from a `finally` block: it silently discards any exception thrown in `try`. And never catch `Exception` just to log and continue. Swallowed exceptions are the hardest production bugs to trace.

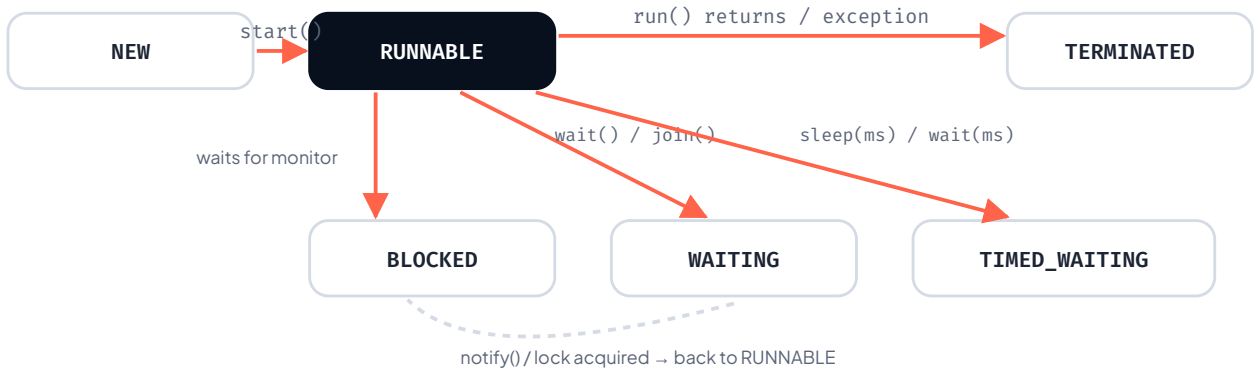
🧵 **Q112** AJ-038 • Intermediate

What is multithreading? How do you create and start a thread?

⚡ **Quick answer**

Multithreading runs several threads concurrently in one process, sharing the heap. Create work as a `Runnable` or `Callable` and run it with `Thread.start()`, an `ExecutorService`, or (Java 21+) virtual threads.

Thread.State (java.lang.Thread.State)



Java

```
// 1. Extend Thread (avoid: wastes your one superclass)
class Worker extends Thread { public void run() { System.out.println("Thread: " + getName()); } }

// 2. Runnable as a lambda
Thread t = new Thread(() -> System.out.println("Runnable task"));
t.start();                          // start() creates a new thread; run() would NOT
t.join();
```


© TechTribers. For personal use only. Do not redistribute.

```java
// 3. ExecutorService + Callable: pooled threads, results via Future
try (ExecutorService pool = Executors.newFixedThreadPool(4)) {    // AutoCloseable: Java 19+
    Future<Integer> f = pool.submit(() -> 21 * 2);
    System.out.println("Callable result: " + f.get());
}

// 4. Java 21 virtual threads: cheap, ideal for blocking I/O
try (var vt = Executors.newVirtualThreadPerTaskExecutor()) {
    vt.submit(() -> System.out.println("Virtual: " + Thread.currentThread().isVirtual()));
}
```

▶ Output

```
Runnable task
Callable result: 42
Virtual: true
```

⚠️ **Interview trap**

Calling `t.run()` instead of `t.start()` just executes the method on the **current** thread; no new thread is created. And calling `start()` twice throws `IllegalThreadStateException`.

💡 **Pro tip**

Prefer `Runnable` / `Callable` plus an executor over extending `Thread`: it separates the task from how it runs, so you can switch from a fixed pool to virtual threads by changing one line.

Q113 AJ-039 • Advanced

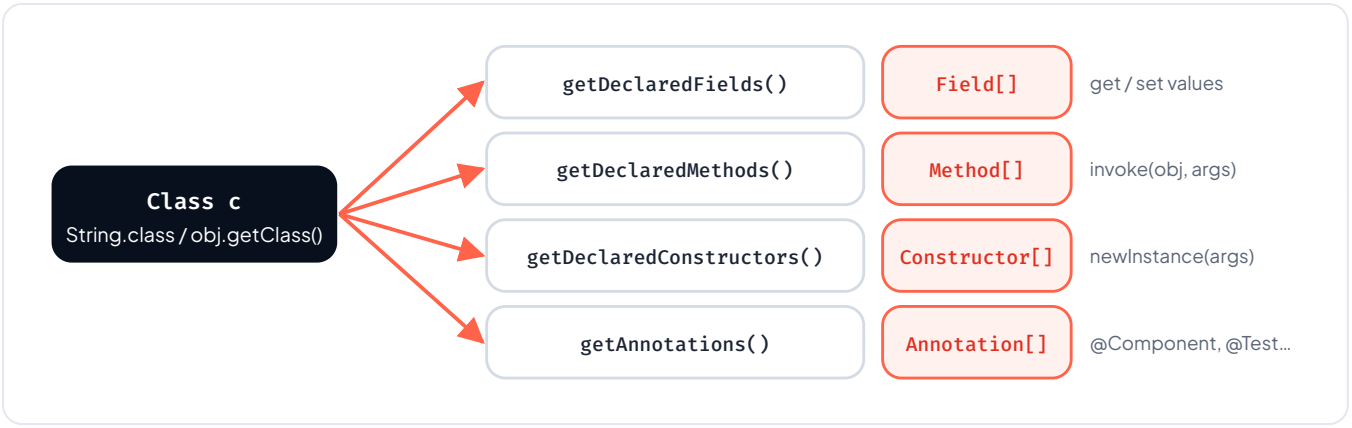
What is Java Reflection? How do you inspect classes and invoke methods?

⚡ **Quick answer**

Reflection lets code inspect and use classes, methods, fields and constructors at runtime through `Class`, `Method`, `Field` and `Constructor`. Frameworks like Spring, Hibernate, Jackson and JUnit are built on it.

Costs to mention: reflective calls are slower and skip compile-time checks, they break refactoring tools, and since JDK 16 (JEP 396/403) `setAccessible(true)` on JDK internals fails with `InaccessibleObjectException` unless the module is explicitly opened.

© TechTribers. For personal use only. Do not redistribute.



Java

```java
record User(String name, int age) {
    private String greet(String p) { return p + ", " + name; }
}

Class<?> c = User.class;
System.out.println(c.getSimpleName() + " " + Arrays.toString(c.getRecordComponents()));

Method m = String.class.getMethod("toUpperCase");
System.out.println(m.invoke("hello"));                          // HELLO

Method greet = c.getDeclaredMethod("greet", String.class);
greet.setAccessible(true);                                      // bypass private
System.out.println(greet.invoke(new User("Asha", 30), "Hi"));

User u = (User) c.getDeclaredConstructor(String.class, int.class).newInstance("Ravi", 28);
System.out.println(u);
```

▶ Output

```
User [java.lang.String name, int age]
HELLO
Hi, Asha
User[name=Ravi, age=28]
```

💡 **Pro tip**

For high-performance dynamic calls, use `MethodHandle` (`java.lang.invoke`). Since JDK 18, core reflection itself is implemented on method handles (JEP 416).

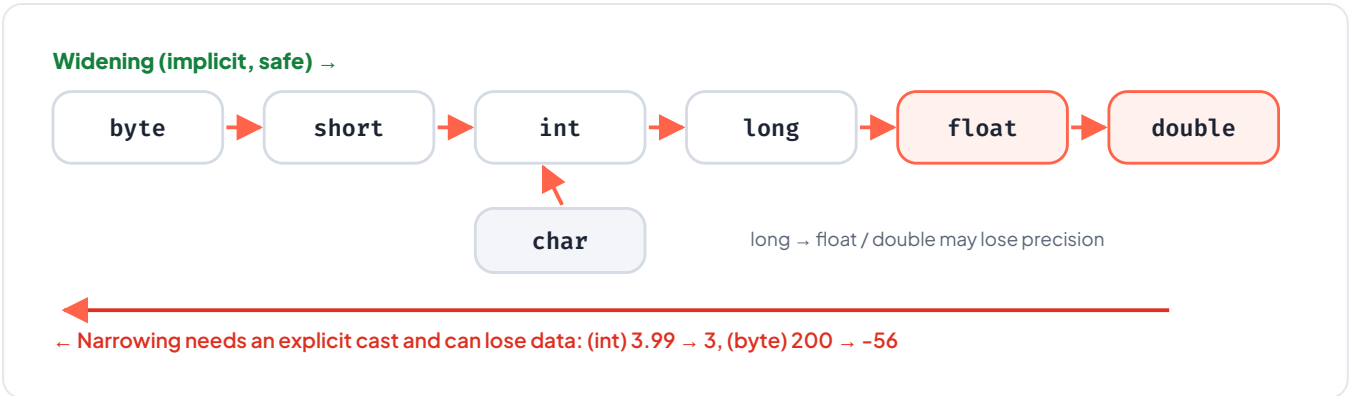
🔄 Q114 AJ-040 • Basic

How do you convert one data type to another? Explain casting and parsing.

⚡ **Quick answer**

Casting converts between numeric types: widening is automatic, narrowing needs an explicit cast and can lose data. **Parsing** turns text into numbers with `Integer.parseInt()` or `valueOf()` .

© TechTribers. For personal use only. Do not redistribute.



Java

```java
int i = 42;
long l = i;                        // widening: automatic
double d = l;                      // 42.0

int fromDouble = (int) 3.99;       // 3   (truncates toward zero)
int fromNeg    = (int) -3.99;      // -3
byte overflow  = (byte) 200;       // -56 (wraps around)
int bigLong    = (int) 3_000_000_000L; // -1294967296

int parsed  = Integer.parseInt("42");      // primitive
Integer boxed = Integer.valueOf("42");     // wrapper (cached -128..127)
String back = String.valueOf(parsed);      // or Integer.toString(parsed)

try { Integer.parseInt("42.5"); }
catch (NumberFormatException e) { System.out.println("Bad number: " + e.getMessage()); }
System.out.println(fromDouble + " " + fromNeg + " " + overflow + " " + bigLong);
```

▶ Output

```
Bad number: For input string: "42.5"
3 -3 -56 -1294967296
```

⚠️ **Interview trap**

Narrowing casts never throw; they silently wrap. For money or counters use `Math.toIntExact(longValue)`, which throws `ArithmeticException` on overflow instead of corrupting data.

© TechTribers. For personal use only. Do not redistribute.

Section 06

I/O, Networking & Date-Time

Reading input, talking over sockets, and handling dates correctly with java.time.

Questions 115–117 · 3 questions



Q115 AJ-041

• Basic

How do you read user input with the `Scanner` class?

⚡ Quick answer

`Scanner` (java.util) parses primitives and strings from an input source such as `System.in`, a `File` or a `String`, using whitespace as the default delimiter.

Java

```
Scanner sc = new Scanner(System.in);
System.out.print("Age: ");
int age = sc.nextInt();
sc.nextLine();                          // consume the leftover newline!
System.out.print("Full name: ");
String name = sc.nextLine();
System.out.println("Hello, " + name + " (" + age + ")");
```

▶ Output

```
Age: 30
Full name: Asha Patil
Hello, Asha Patil (30)
```

⚠️ Interview trap

`nextInt()` leaves the newline in the buffer, so the following `nextLine()` returns an empty string immediately. Add a `nextLine()` after it, or read every line with `nextLine()` and parse it.

💡 Pro tip

`Scanner` is slow for large input because it uses regex. In competitive programming or log processing, use `BufferedReader.readLine()` plus `Integer.parseInt`. Also don't close a `Scanner` on `System.in` if you need to read again later: it closes `System.in` too.

© TechTribers. For personal use only. Do not redistribute.

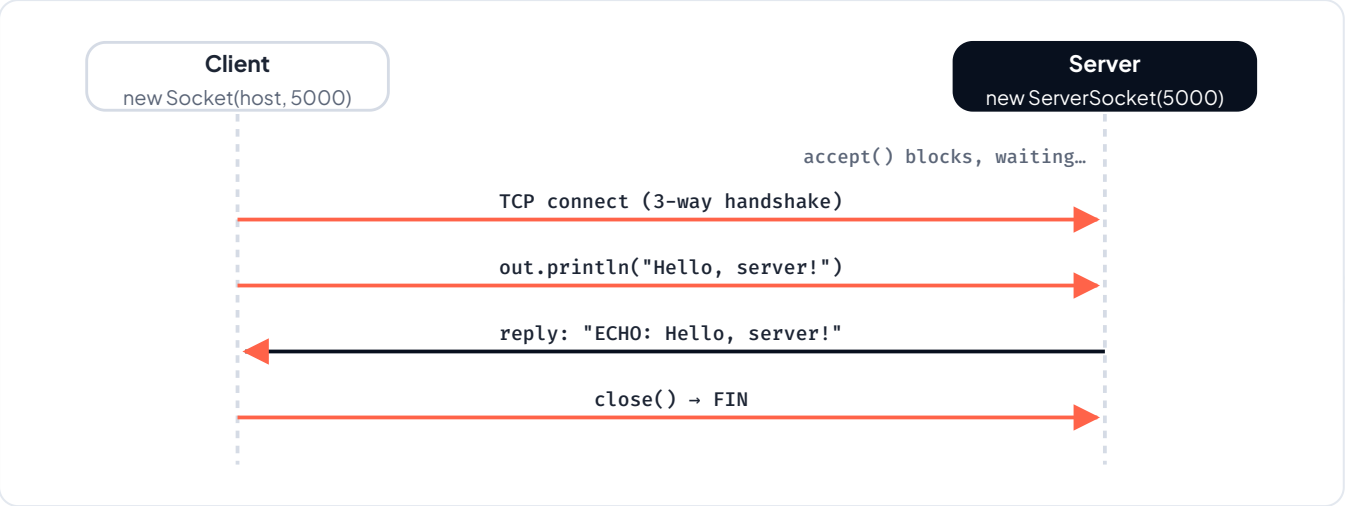
Q116 AJ-042 • Intermediate

How do you create a simple client-server application with sockets?

⚡ Quick answer

The server opens a `ServerSocket` on a port and blocks on `accept()`. The client connects with `new Socket(host, port)`. Both sides then exchange data through the socket's input and output streams over TCP.

Improvements over the classic textbook version: try-with-resources closes everything even on errors, explicit UTF-8 avoids encoding bugs, and a thread per client means one slow client doesn't block all the others.



Java

```
// Server: echoes each line, one virtual thread per client (Java 21+)
try (ServerSocket server = new ServerSocket(5000)) {
    while (true) {
        Socket client = server.accept();
        Thread.startVirtualThread(() -> {
            try (client;
                 var in  = new BufferedReader(new InputStreamReader(client.getInputStream(),
UTF_8));
                 var out = new PrintWriter(client.getOutputStream(), true, UTF_8)) {
                String line;
                while ((line = in.readLine()) != null) out.println("ECHO: " + line);
            } catch (IOException e) { e.printStackTrace(); }
        });
    }
}

// Client
try (Socket s = new Socket("localhost", 5000);
     var out = new PrintWriter(s.getOutputStream(), true, UTF_8);
     var in  = new BufferedReader(new InputStreamReader(s.getInputStream(), UTF_8))) {
    out.println("Hello, server!");
    System.out.println(in.readLine());
}
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
ECHO: Hello, server!
```

💡 **Pro tip**

For HTTP, don't use raw sockets: use `java.net.http.HttpClient` (Java 11+), which supports HTTP/2, async calls and timeouts out of the box.

Q117 AJ-043 • Intermediate

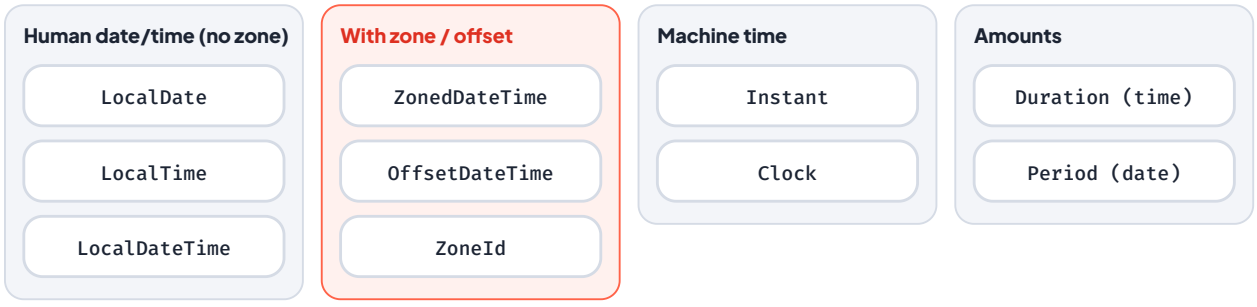
How do you work with dates in Java? Why not `Date` and `SimpleDateFormat`?

⚡ **Quick answer**

Use the `java.time` API (Java 8+): `LocalDate`, `LocalDateTime`, `ZonedDateTime`, `Instant`, `Duration`, `Period`, formatted with `DateTimeFormatter`. The legacy `Date`, `Calendar` and `SimpleDateFormat` are mutable and error-prone.

Why the legacy API is avoided:

- `SimpleDateFormat` is **not thread-safe**. Shared as a `static` field, it produces corrupted dates under load.
- `Date` is mutable, and `Calendar` months start at **0** (January = 0).
- `java.time` types are immutable, thread-safe, and separate human dates from machine instants.



All immutable and thread-safe. Format/parse with DateTimeFormatter (also thread-safe).

Java

```java
LocalDate today = LocalDate.of(2026, 10, 14);
var fmt = DateTimeFormatter.ofPattern("dd MMM yyyy", Locale.ENGLISH);

System.out.println(today.format(fmt) + " is a " + today.getDayOfWeek());
System.out.println(today.plusDays(30).format(fmt));
System.out.println(Period.between(LocalDate.of(2026, 1, 1), today));

ZonedDateTime ist = ZonedDateTime.of(2026, 10, 14, 9, 30, 0, 0, ZoneId.of("Asia/Kolkata"));
System.out.println(ist.withZoneSameInstant(ZoneId.of("Europe/Copenhagen")).toLocalTime());
```


© TechTribers. For personal use only. Do not redistribute.

```
LocalDate parsed = LocalDate.parse("2026-12-31");                // ISO by default
System.out.println(ChronoUnit.DAYS.between(today, parsed) + " days to year end");
```

▶ Output

```
14 Oct 2026 is a WEDNESDAY
13 Nov 2026
P9M13D
06:00
78 days to year end
```

💡 **Pro tip**

Store timestamps as `Instant` (UTC) in databases and events; convert to a `ZonedDateTime` only at the edges, for display.

Section 07

Java GUI & Legacy APIs

AWT, Swing, JavaFX and Applets: still asked in interviews, mostly to check you know what is current and what is obsolete.

Questions 118–121 · 4 questions



Q118 AJ-044

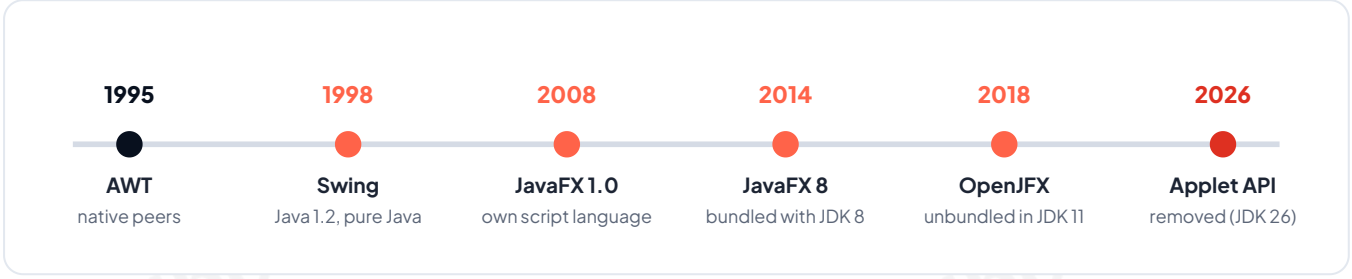
• Basic

What is AWT? How do you create a simple window with `Frame`?

⚡ **Quick answer**

AWT (Abstract Window Toolkit, Java 1.0) is Java's original GUI toolkit. Its components are **heavyweight**: each one wraps a native OS widget (a "peer"), so they look different on each platform.

Today AWT matters mostly as the foundation: Swing is built on top of it, and the AWT event model (`ActionListener`, `WindowListener`) is still used by Swing.



© TechTribers. For personal use only. Do not redistribute.

```java
import java.awt.*;
import java.awt.event.*;

public class SimpleWindow {
    public static void main(String[] args) {
        Frame frame = new Frame("TechTribers");
        frame.add(new Label("Hello, AWT!", Label.CENTER));
        frame.setSize(300, 200);
        frame.addWindowListener(new WindowAdapter() {     // without this, [X] does nothing
            @Override public void windowClosing(WindowEvent e) { frame.dispose(); }
        });
        frame.setVisible(true);
    }
}
```

⚠️ **Interview trap**

A bare `Frame` ignores the close button: clicking [X] does nothing until you add a `WindowListener`. This is a favourite "what's wrong with this code?" question.

Q119 AJ-045 • Basic

How do you create a simple Swing application with a button and event handling?

⚡ **Quick answer**

Swing (`javax.swing`, Java 1.2) draws **lightweight** components in pure Java on top of AWT. Create a `JFrame`, add a `JButton`, register an `ActionListener`, and build the UI on the Event Dispatch Thread.

```java
import javax.swing.*;

public class SimpleSwingApp {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {                // build UI on the EDT
            JFrame frame = new JFrame("TechTribers");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

            JButton button = new JButton("Click me");
            button.addActionListener(e ->
                    JOptionPane.showMessageDialog(frame, "Button clicked!"));

            frame.add(button);
            frame.pack();
            frame.setLocationRelativeTo(null);            // centre on screen
            frame.setVisible(true);
        });
```


© TechTribers. For personal use only. Do not redistribute.

```
    }
}
```

⚠️ **Interview trap**

Swing is **single-threaded**. Creating or updating components outside the Event Dispatch Thread causes random repaint glitches and deadlocks, and slow work inside a listener freezes the UI. Use `SwingUtilities.invokeLater` for UI updates and `SwingWorker` for background tasks.

💡 **Pro tip**

IntelliJ IDEA and many trading and banking desktop tools are still built on Swing, so the EDT rules are not just theory.

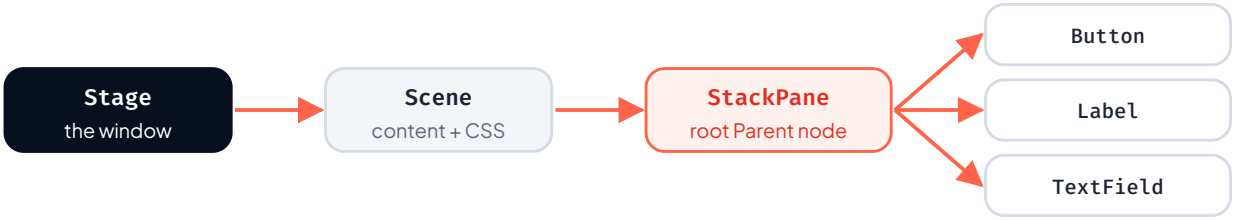
🎨 **Q120** AJ-046 • Intermediate

What is JavaFX? How do you create a simple JavaFX app with a button?

⚡ **Quick answer**

JavaFX is the modern Java GUI toolkit with a scene graph, CSS styling, FXML layouts, animation and hardware-accelerated rendering. Since Java 11 it ships separately as **OpenJFX**, added as a Maven or Gradle dependency.

Lifecycle: `launch()` calls `init()` (on a launcher thread), then `start(Stage)` on the FX Application Thread, then `stop()` when the app exits. The same single-thread rule as Swing applies: update the UI with `Platform.runLater()`.



Java

```
public class SimpleFxApp extends Application {

    @Override
    public void start(Stage stage) {                    // runs on the JavaFX Application Thread
        Label status = new Label("Ready");
        Button button = new Button("Click me");
        button.setOnAction(e -> status.setText("Button clicked!"));

        VBox root = new VBox(12, button, status);
        root.setStyle("-fx-padding: 24; -fx-alignment: center;");
        stage.setScene(new Scene(root, 300, 200));
        stage.setTitle("TechTribers");
```


© TechTribers. For personal use only. Do not redistribute.

```
        stage.show();
    }

    public static void main(String[] args) { launch(args); }
}
```

⚠️ **Interview trap**

"JavaFX is part of the JDK" was true only for Oracle JDK 8 through 10. On JDK 11+ you need the `org.openjfx` dependencies (for example `javafx-controls`), or a JDK build that bundles them.

🧑 **Q121** AJ-047 • Basic

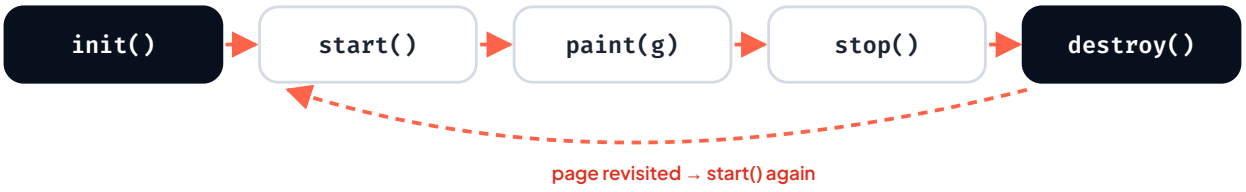
What is a Java Applet? Is it still used?

⚡ **Quick answer**

An applet was a Java program embedded in a web page and run by a browser plug-in, with an `init` → `start` → `paint` → `stop` → `destroy` lifecycle. It is obsolete: browsers dropped the plug-in, and the Applet API has been removed from the JDK.

The timeline to quote:

- Browsers removed NPAPI plug-in support around 2015–2017.
- **Java 9** deprecated the Applet API; **Java 11** removed the browser plug-in, Java Web Start and `appletviewer`.
- **Java 17** marked the API for removal (JEP 398), and **JDK 26** removed it (JEP 504).



Java

```
// Historical example: compiles only on JDKs that still ship java.applet (JDK 25 and earlier)
import java.applet.Applet;
import java.awt.Graphics;

public class HelloApplet extends Applet {
    @Override public void init()  { /* one-time setup */ }
    @Override public void paint(Graphics g) { g.drawString("Hello, Applet!", 20, 30); }
}
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> If asked "what replaced applets?", say: web front-ends (JavaScript/TypeScript) talking to Java back-ends over HTTP. For desktop delivery, JavaFX or Swing packaged with `jpackage`.

Series progress

121 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 3 starts at Q122.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.